

Implementing a Secure Firmware Update Mechanism for IoT Devices

MOHAMMAD HAJI

Mohammadhaji499@gmail.com

Abstract

The increasing use of Internet of Things (IoT) devices in homes, industries, healthcare, and smart cities has made firmware security an important cybersecurity concern. Firmware updates are necessary to fix vulnerabilities and improve device functionality, but insecure update mechanisms can expose devices to unauthorized modifications, malware installation, and downgrade attacks. This project implements a secure firmware update mechanism for IoT devices using RSA-based digital signatures, SHA-256 integrity verification, HTTPS/TLS communication, role-based access control (RBAC), rollback protection, audit logging, and recovery mode functionality. The implementation was developed and tested in a controlled Kali Linux environment using open-source tools. Testing demonstrated successful detection of tampered firmware, prevention of unauthorized updates, secure update delivery, and recovery from failed update scenarios.

1. Introduction

Firmware acts as the low-level software that controls the operation of IoT devices. As IoT adoption continues to grow, attackers increasingly target firmware because compromising firmware can provide persistent and privileged access to a device. Traditional update mechanisms that lack authentication and integrity validation are vulnerable to firmware tampering, unauthorized modifications, and downgrade attacks.

To address these risks, organizations implement secure firmware update frameworks that verify the authenticity and integrity of firmware before installation. This project focuses on designing and implementing a practical secure firmware update workflow that incorporates cryptographic verification, secure communication, access control, audit logging, rollback protection, and recovery mechanisms.

The project was implemented using Python, Flask, OpenSSL, and various security validation techniques to simulate a real-world IoT firmware update environment.

2. Problem Statement

Many IoT devices rely on insecure firmware update processes that do not properly validate firmware authenticity or integrity. As a result, attackers may distribute malicious firmware, modify legitimate updates, intercept update traffic, or force devices to install older vulnerable firmware versions.

The challenge is to design a secure firmware update mechanism that ensures only trusted firmware is installed while protecting update communications, restricting access to authorized users, maintaining audit logs, and supporting recovery from failed update attempts.

4. Project Objectives

The primary objectives of this project are:

- Implement RSA-based digital signing for firmware authentication.
- Use SHA-256 hashing for firmware integrity verification.
- Develop a secure firmware update server using HTTPS/TLS.
- Implement authentication mechanisms for firmware distribution.
- Simulate secure firmware verification before installation.
- Detect and reject tampered firmware updates.
- Implement rollback protection against downgrade attacks.
- Develop a firmware recovery mechanism for failed updates.
- Implement Role-Based Access Control (RBAC) for update management.
- Generate audit logs for security monitoring and accountability.
- Align the implementation with NIST IoT and IoT Security Foundation recommendations.

5. Technologies Used

This project was implemented using various open-source technologies, tools, and frameworks to ensure secure firmware distribution, verification, access control, and testing. The selected technologies provide cryptographic security, secure communication, automation capabilities, and effective monitoring for IoT firmware update management.

5.1 Kali Linux

Kali Linux was used as the primary development and testing environment. It provided the required security tools, networking utilities, Python environment, and OpenSSL support needed for implementing and validating the secure firmware update mechanism.

5.2 Python 3

Python 3 was used to develop automation scripts for rollback protection, firmware recovery, and role-based access control (RBAC). Python provided a simple and efficient platform for implementing security validation workflows.

5.3 OpenSSL

OpenSSL was used for cryptographic operations including RSA key generation, digital signature creation, signature verification, and TLS certificate generation. It enabled secure firmware authentication and integrity validation.

5.4 Flask

Flask was used to develop the secure firmware update server. It provided REST endpoints for firmware distribution, update status checks, authentication handling, and audit logging.

5.5 Cryptography Library

The Python Cryptography library was used to support cryptographic operations and security-related functions within the project environment.

5.6 PyOpenSSL

PyOpenSSL was used to enable SSL/TLS functionality and support secure communication between the simulated IoT device and the update server.

5.7 HTTPS/TLS

HTTPS secured firmware transmission between the update server and IoT devices. TLS encryption ensured confidentiality and integrity of update communications and protected against unauthorized interception.

5.8 RSA-2048

RSA-2048 public key cryptography was implemented for firmware signing and signature verification. The private key was used for signing firmware images, while the public key was used to verify authenticity before installation.

5.9 SHA-256

SHA-256 hashing was used to generate cryptographic fingerprints of firmware files. Any modification to the firmware resulted in a different hash value, enabling tampering detection.

5.10 VirtualBox

VirtualBox was used to create the controlled testing environment in which Kali Linux and all project components were deployed and tested.

Table 5.1: Technologies Used

Technology	Purpose
Kali Linux	Development and testing environment
Python 3	Automation scripts and validation logic
OpenSSL	RSA signing, hashing, and certificate generation
Flask	Secure firmware update server
Cryptography Library	Cryptographic operations
PyOpenSSL	SSL/TLS support
HTTPS/TLS	Secure communication
RSA-2048	Firmware authentication
SHA-256	Firmware integrity verification
VirtualBox	Virtualized lab environment

6. SYSTEM ARCHITECTURE

6.1 Architecture Overview

The Secure Firmware Update Mechanism for IoT Devices was designed to ensure that firmware updates are distributed, verified, and installed securely. The architecture combines cryptographic verification, secure communication, access control, rollback protection, recovery mechanisms, and audit logging to protect IoT devices from unauthorized firmware execution and malicious updates.

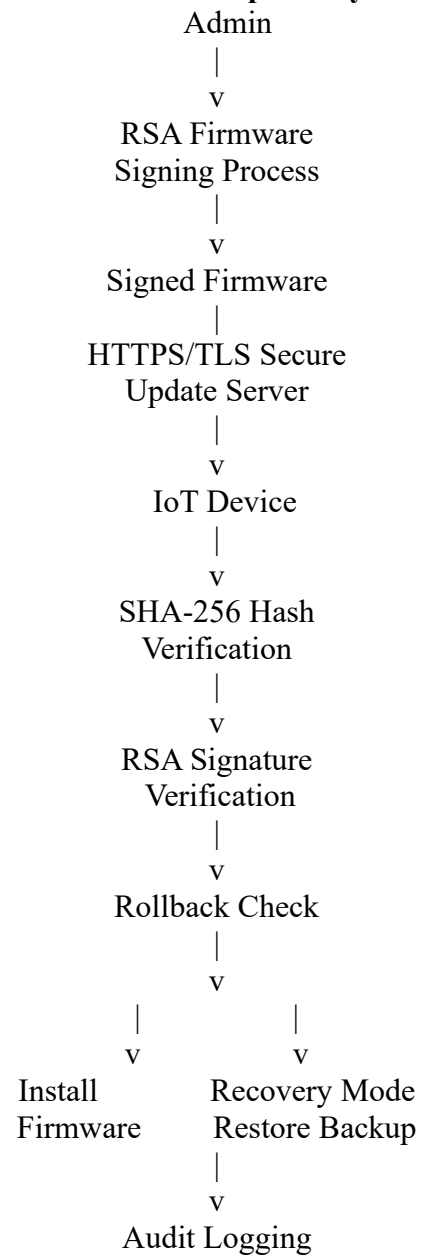
The system follows a layered security approach where firmware authenticity is verified before installation and all update activities are monitored through logging and access control mechanisms.

The architecture consists of the following major components:

- Firmware Signing System
- Secure Update Server
- Firmware Verification Module
- Rollback Protection Mechanism
- Recovery Mode Module
- Role-Based Access Control (RBAC)
- Audit Logging System

6.2 Overall System Architecture

Figure 6.1: Secure Firmware Update System Architecture



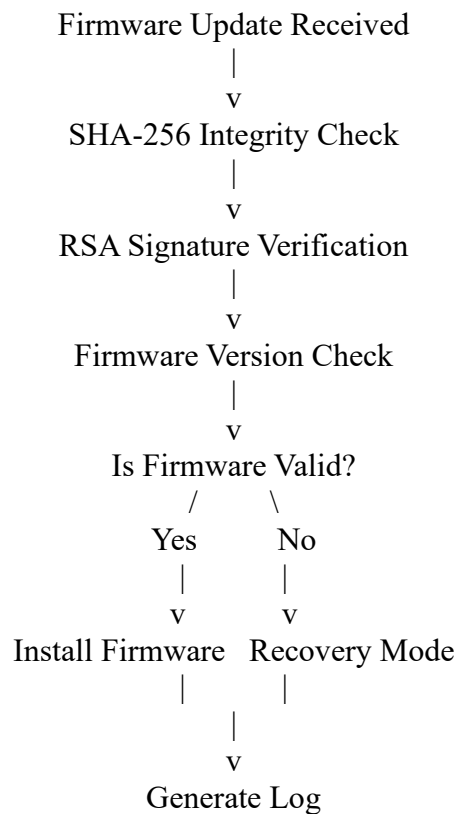
Explanation

The administrator generates and signs firmware using an RSA private key. The signed firmware is uploaded to a secure update server protected by HTTPS/TLS encryption. IoT devices download firmware updates from the server and perform SHA-256 integrity verification followed by RSA signature verification.

Before installation, rollback protection validates the firmware version to prevent downgrade attacks. If all checks succeed, the firmware is installed. If verification fails, the recovery module restores a trusted backup firmware image. All activities are recorded through the audit logging system.

6.3 Secure Firmware Verification Workflow

Figure 6.2: Firmware Verification Workflow



Explanation

When a firmware update is received, the device first performs SHA-256 integrity validation. The firmware signature is then verified using the RSA public key. After successful verification, firmware version validation is performed to detect rollback attacks.

If all validation checks pass, the firmware is installed successfully. If any verification step fails, the recovery mechanism restores the backup firmware and records the event in the audit logs.

6.4 Security Components

Firmware Signing System

Uses RSA-2048 digital signatures to verify firmware authenticity and ensure updates originate from a trusted source.

Secure Update Server

Provides encrypted firmware distribution through HTTPS/TLS and implements token-based authentication.

Rollback Protection

Prevents installation of older vulnerable firmware versions through version comparison logic.

Recovery Mode

Restores backup firmware in case of failed verification or update corruption.

RBAC

Restricts firmware management actions according to user roles including Administrator, Operator, and Viewer.

Audit Logging

Records update requests, firmware downloads, unauthorized access attempts, and recovery events for security monitoring.

7. Cryptographic Implementation

7.1 Firmware Creation

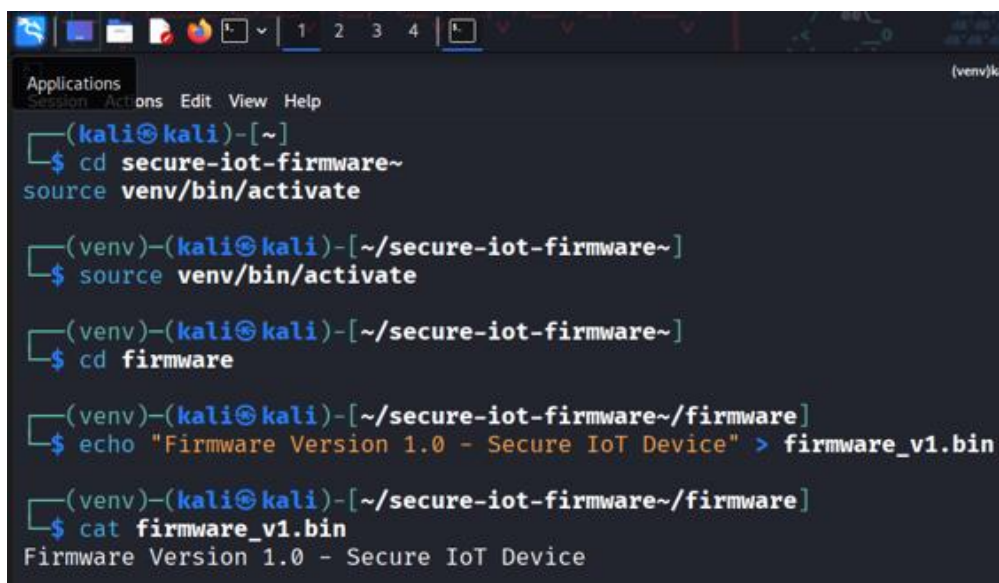
A simulated firmware file was created to represent the software image that would be deployed on an IoT device. The firmware file was used throughout the project to test digital signing, integrity verification, tampering detection, rollback protection, and recovery mechanisms.

The firmware was created using the following command:

```
echo "Firmware Version 1.0 - Secure IoT Device" > firmware/firmware_v1.bin
```

The created firmware file served as the base firmware image for all cryptographic validation and security testing activities performed during the project.

Figure 7.1: Firmware Creation Process



```
Applications
Session Actions Edit View Help

(kali㉿kali)-[~]
$ cd secure-iot-firmware~
source venv/bin/activate

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ source venv/bin/activate

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ cd firmware

(venv)-(kali㉿kali)-[~/secure-iot-firmware~/firmware]
$ echo "Firmware Version 1.0 - Secure IoT Device" > firmware_v1.bin

(venv)-(kali㉿kali)-[~/secure-iot-firmware~/firmware]
$ cat firmware_v1.bin
Firmware Version 1.0 - Secure IoT Device
```

7.2 RSA Key Generation

To ensure firmware authenticity, RSA-2048 public key cryptography was implemented. RSA was selected because it provides strong digital signature capabilities and is widely used for firmware authentication in secure update systems.

The RSA private key was generated using OpenSSL:

```
openssl genrsa -out keys/private.pem 2048
```

The corresponding public key was generated using:

```
openssl rsa -in keys/private.pem -pubout -out keys/public.pem
```

The generated keys perform the following functions:

Key	Purpose
private.pem	Firmware signing
public.pem	Signature verification

The private key represents the manufacturer's signing key, while the public key simulates the verification key stored on IoT devices.

Figure 7.2: RSA Key Generation Using OpenSSL

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl genrsa -out keys/private.pem 2048

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl rsa -in keys/private.pem -pubout -out keys/public.pem
writing RSA key

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ ls keys
private.pem  public.pem
```

7.3 SHA-256 Verification

SHA-256 hashing was implemented to verify firmware integrity. A cryptographic hash provides a unique fingerprint for the firmware file. Any modification to the firmware results in a completely different hash value.

The following command was used:

```
sha256sum firmware/firmware_v1.bin
```

Generated Hash:

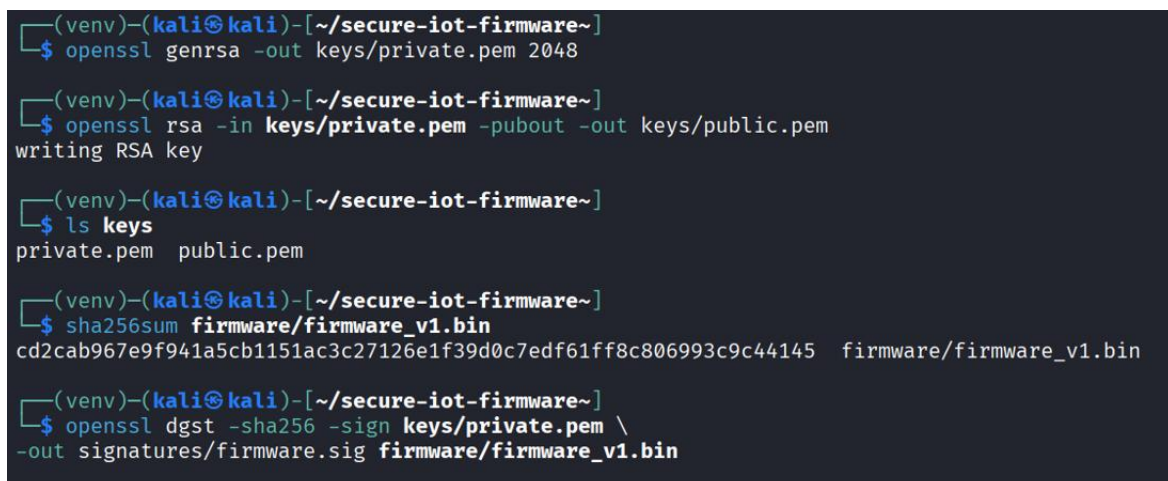
```
cd2cab967e9f941a5cb1151ac3c27126e1f39d0c7edf61ff8c806993c9c44145
```

SHA-256 verification provides protection against:

- Firmware tampering
- File corruption
- Unauthorized modifications
- Integrity violations

If the firmware content changes, the generated hash value also changes, allowing the system to detect unauthorized modifications.

Figure 7.3: SHA-256 Hash Generation



```
(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ openssl genrsa -out keys/private.pem 2048

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ openssl rsa -in keys/private.pem -pubout -out keys/public.pem
writing RSA key

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ ls keys
private.pem  public.pem

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ sha256sum firmware/firmware_v1.bin
cd2cab967e9f941a5cb1151ac3c27126e1f39d0c7edf61ff8c806993c9c44145  firmware/firmware_v1.bin

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -sign keys/private.pem \
-out signatures/firmware.sig firmware/firmware_v1.bin
```

7.4 Digital Signing

Digital signing was implemented using the RSA private key to ensure firmware authenticity and trusted origin verification.

The firmware signing process was performed using:

```
openssl dgst -sha256 -sign keys/private.pem \
-out signatures/firmware.sig firmware/firmware_v1.bin
```

The generated signature file:

firmware.sig

was stored separately and used during firmware verification.

Firmware verification was performed using the RSA public key:

```
openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
```

Verification Result:

Verified OK

A successful verification confirms:

- Firmware authenticity
- Trusted firmware source
- Firmware integrity
- Secure execution eligibility

To validate the security mechanism, a tampering simulation was performed by modifying the firmware after signing:

```
echo "Malicious Backdoor Injected" >> firmware/firmware_v1.bin
```

After modification, signature verification was executed again.

Result:

Verification

failure

bad signature

This demonstrated successful detection of unauthorized firmware modification.

Figure 7.4: Successful Signature Verification

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ sha256sum firmware/firmware_v1.bin
cd2cab967e9f941a5cb1151ac3c27126e1f39d0c7edf61ff8c806993c9c44145  firmware/firmware_v1.bin

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -sign keys/private.pem \
-out signatures/firmware.sig firmware/firmware_v1.bin

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
Verified OK
```

Figure 7.5: Tampered Firmware Detection

```
(venv)-(kali@kali)-[~/secure-iot-firmware~/firmware]
$ cd ..

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
Verification failure
40D70B05187F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
40D70B05187F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:
```

Result Analysis

The cryptographic implementation successfully demonstrated secure firmware authentication and integrity validation. RSA digital signatures ensured that only trusted firmware could be accepted, while SHA-256 hashing enabled reliable detection of firmware tampering and unauthorized modifications. The testing results confirmed that the implemented security controls effectively protected the firmware update process against integrity and authenticity attacks.

8. Secure Update Server

The secure update server was implemented to simulate the secure distribution of firmware updates to IoT devices. The server was designed to provide encrypted communication, firmware delivery, update status checking, authentication, and audit logging functionality.

A Flask-based web application was developed and configured to operate over HTTPS/TLS. This approach ensured that firmware updates were transmitted through an encrypted communication channel, reducing the risk of interception, modification, or unauthorized access.

8.1 Objectives

The objectives of the secure update server implementation were:

- Secure firmware distribution
- Encrypted communication using HTTPS/TLS
- Authentication of update requests
- Firmware update status monitoring
- Audit logging of update activities
- Simulation of OTA (Over-The-Air) update delivery

8.2 TLS Certificate Generation

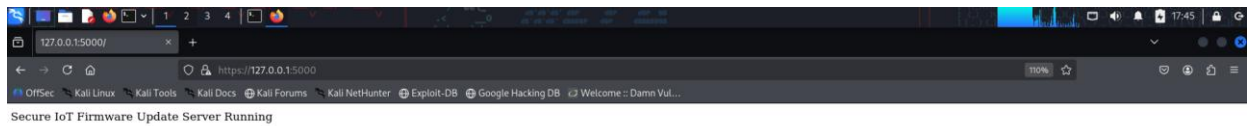
To enable secure HTTPS communication, self-signed TLS certificates were generated using OpenSSL.

The following command was used:

```
openssl req -x509 -newkey rsa:2048 \
-keyout key.pem -out cert.pem \
-days 365 -nodes
```

The certificates were used in a controlled laboratory environment for testing secure firmware delivery.

Figure 8.2: Flask Secure Update Server Running



8.4 Firmware Update Endpoints

Root Endpoint

Purpose:

- Verify server availability
- Confirm HTTPS connectivity

Update Check Endpoint

/check-update

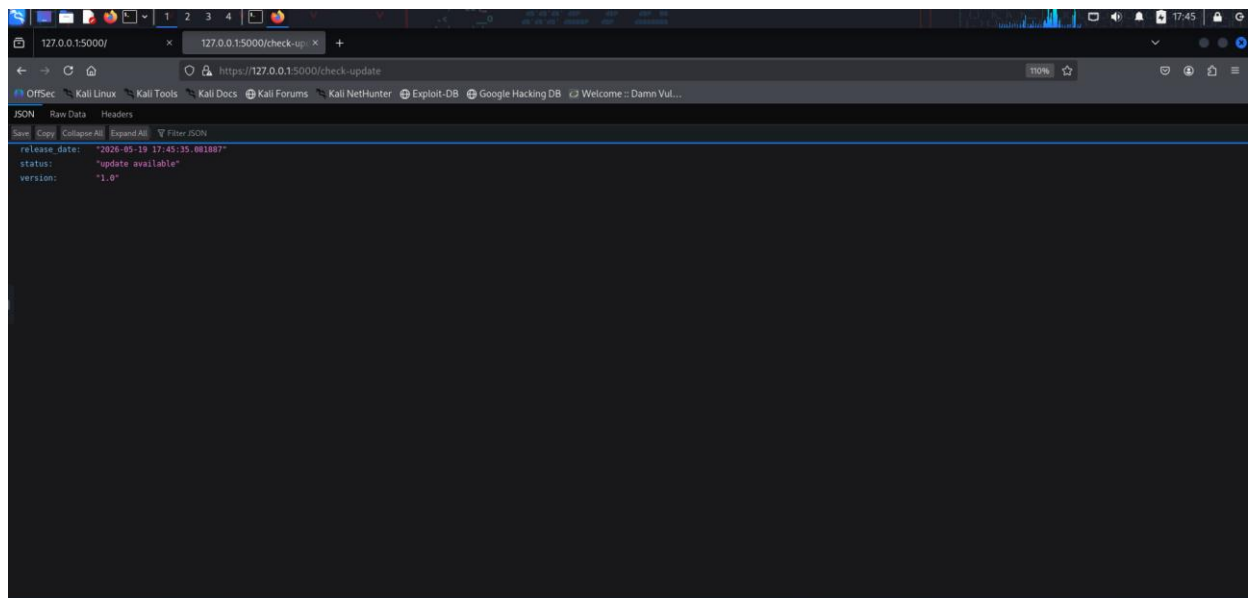
Purpose:

- Simulate firmware update requests
- Provide update availability information

Example Response:

```
{
  "version": "1.0",
  "status": "update available"
}
```

Figure 8.3: Update Check API Response



Secure Firmware Download Endpoint

/download-firmware

Purpose:

- Deliver firmware files securely
- Validate authorization requests
- Prevent unauthorized firmware downloads

8.5 Authentication Mechanism

Token-based authentication was implemented using HTTP Authorization headers.

Authorized request:

```
curl -k -H "Authorization: secureiottoken" \
https://127.0.0.1:5000/download-firmware
```

The request successfully returned the firmware file.

Unauthorized request:

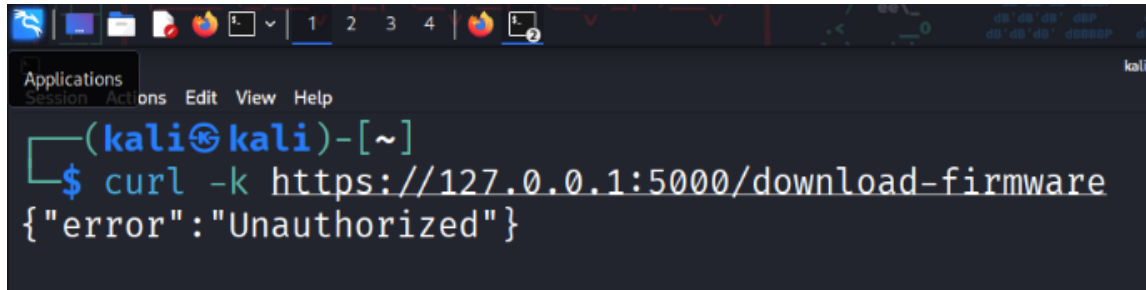
```
curl -k https://127.0.0.1:5000/download-firmware
```

Result:

Unauthorized

This demonstrated successful enforcement of access control policies.

Figure 8.4: Unauthorized Access Attempt

A screenshot of a Kali Linux terminal window. The window has a title bar with 'Applications' and a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal prompt is '(kali㉿kali)-[~]'. The user has entered the command 'curl -k https://127.0.0.1:5000/download-firmware'. The output of the command is a JSON object: '{"error": "Unauthorized"}'.

```
(kali㉿kali)-[~]  
$ curl -k https://127.0.0.1:5000/download-firmware  
{"error": "Unauthorized"}
```

8.6 Audit Logging

Audit logging was implemented to record security-relevant events including:

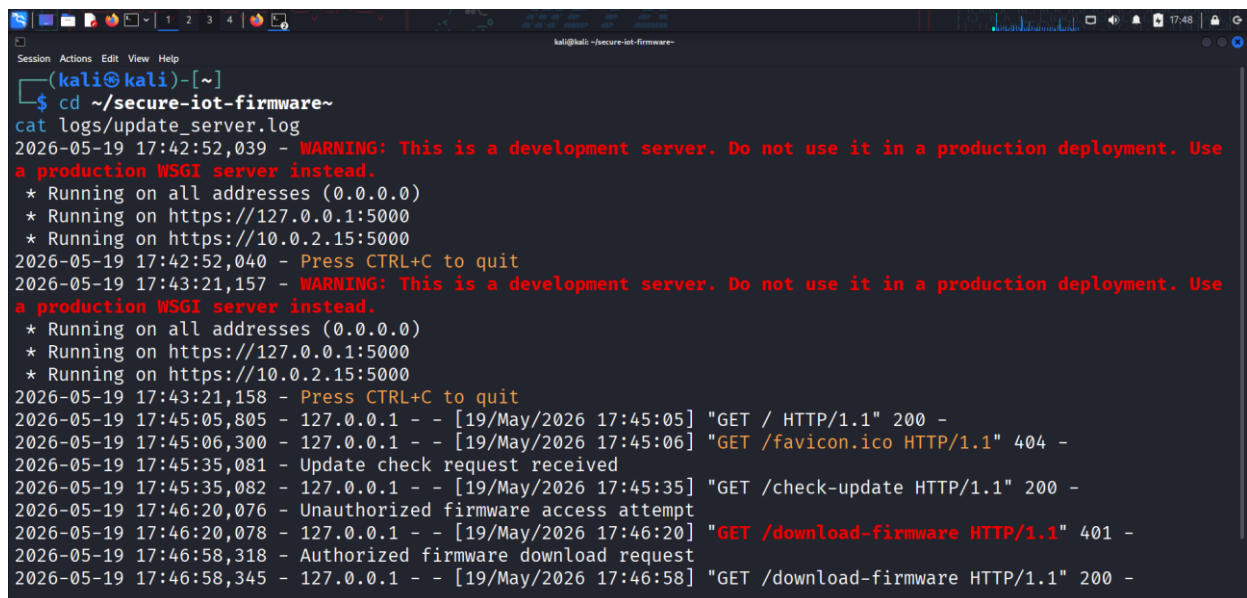
- Update check requests
- Firmware downloads requests
- Unauthorized access attempts
- General server activity

The generated logs provide visibility into update operations and support security monitoring.

Example Logged Events:

Unauthorized	firmware	access	attempt
Update	check	request	received
Authorized firmware download request			

Figure 8.5: Audit Logging Output

A screenshot of a terminal window titled 'kali@kali: ~/secure-iot-firmware-'. The terminal shows the execution of 'cat logs/update_server.log'. The output consists of several lines of log entries. It starts with a warning message in red: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' followed by status messages: '* Running on all addresses (0.0.0.0)', '* Running on https://127.0.0.1:5000', and '* Running on https://10.0.2.15:5000'. The log then shows a series of HTTP requests and responses, including a 404 for '/favicon.ico', a 200 for '/check-update', a 401 for '/download-firmware', and a 200 for '/download-firmware'. The terminal window has a dark background and standard Linux terminal icons at the top.

```
Session Actions Edit View Help
kali@kali: ~/secure-iot-firmware-
(kali@kali)-[~]
$ cd ~/secure-iot-firmware~
cat logs/update_server.log
2026-05-19 17:42:52,039 - WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on https://127.0.0.1:5000
* Running on https://10.0.2.15:5000
2026-05-19 17:42:52,040 - Press CTRL+C to quit
2026-05-19 17:43:21,157 - WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on https://127.0.0.1:5000
* Running on https://10.0.2.15:5000
2026-05-19 17:43:21,158 - Press CTRL+C to quit
2026-05-19 17:45:05,805 - 127.0.0.1 - - [19/May/2026 17:45:05] "GET / HTTP/1.1" 200 -
2026-05-19 17:45:06,300 - 127.0.0.1 - - [19/May/2026 17:45:06] "GET /favicon.ico HTTP/1.1" 404 -
2026-05-19 17:45:35,081 - Update check request received
2026-05-19 17:45:35,082 - 127.0.0.1 - - [19/May/2026 17:45:35] "GET /check-update HTTP/1.1" 200 -
2026-05-19 17:46:20,076 - Unauthorized firmware access attempt
2026-05-19 17:46:20,078 - 127.0.0.1 - - [19/May/2026 17:46:20] "GET /download-firmware HTTP/1.1" 401 -
2026-05-19 17:46:58,318 - Authorized firmware download request
2026-05-19 17:46:58,345 - 127.0.0.1 - - [19/May/2026 17:46:58] "GET /download-firmware HTTP/1.1" 200 -
```

8.7 Result Analysis

The secure update server successfully demonstrated encrypted firmware distribution, secure communication, authentication enforcement, and activity monitoring. HTTPS/TLS protected firmware transmissions, while token-based authentication restricted access to authorized devices only. Audit logging provided an effective mechanism for monitoring update activities and detecting unauthorized access attempts.

The implementation successfully fulfilled the secure update server requirements specified in the project objectives.

9. Firmware Verification & Tampering Detection

9.1 Overview

Firmware verification is a critical security mechanism that ensures only authentic and trusted firmware is installed on IoT devices. In this project, firmware authenticity was verified using RSA digital signatures, while firmware integrity was validated using SHA-256 hashing.

The verification process was designed to detect unauthorized modifications, corruption, or malicious tampering before firmware installation.

9.2 Firmware Verification Process

The verification process consisted of the following stages:

1. Firmware creation
2. RSA digital signing
3. SHA-256 integrity validation
4. Signature verification using the public key
5. Firmware acceptance or rejection

The verification workflow ensured that only firmware signed by an authorized source could be executed by the IoT device.

Verification Command

The firmware signature was verified using:

```
openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
```


Successful Verification Result:

Verified OK

The result confirmed:

- Firmware authenticity
- Firmware integrity
- Trusted firmware source
- Secure update validation

Figure 9.1: Successful Firmware Verification

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
Verified OK
```

9.3 Tampering Simulation

To validate the effectiveness of the verification mechanism, a firmware tampering attack was simulated. After the firmware had been signed, unauthorized content was added to the firmware file.

Tampering Command:

```
echo "Malicious Backdoor Injected" >> firmware/firmware_v1.bin
```

Modified Firmware:

Firmware	Version	1.0	-	Secure	IoT	Device
Malicious Backdoor Injected						

This modification simulated a malicious actor attempting to insert unauthorized functionality into a legitimate firmware update.

9.4 Tampered Firmware Verification

After modification, the original signature verification process was executed again.

Verification Command:

```
openssl dgst -sha256 -verify keys/public.pem \
-signture signatures/firmware.sig firmware/firmware_v1.bin
```

Observed Result:

```
Verification failure
bad signature
```

The verification mechanism successfully detected that the firmware content no longer matched the original signed version.

Figure 9.2: Tampered Firmware Detection



```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
└─$ openssl dgst -sha256 -verify keys/public.pem \
-signture signatures/firmware.sig firmware/firmware_v1.bin
Verification failure
40D70B05187F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
40D70B05187F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:
```

9.5 Security Analysis

The tampering simulation demonstrated that the implemented cryptographic controls were capable of detecting unauthorized firmware modifications.

The system successfully:

- Detected firmware tampering
- Rejected modified firmware
- Prevented unauthorized firmware execution
- Protected firmware integrity
- Enforced trusted firmware validation

This behavior closely reflects real-world secure firmware update systems where modified or malicious firmware is automatically rejected before installation.

9.6 Testing Results

Test Scenario	Expected Result	Actual Result	Status
Valid Firmware Verification	Firmware Accepted	Verified OK	Pass
Tampered Firmware Verification	Firmware Rejected	Verification Failure	Pass
Integrity Validation	Hash Verification Successful	Successful	Pass
Unauthorized Modification Detection	Tampering Detected	Detected Successfully	Pass

9.7 Conclusion

The firmware verification mechanism successfully validated firmware authenticity and integrity using RSA digital signatures and SHA-256 hashing. The tampering test confirmed that unauthorized modifications were detected immediately, ensuring that only trusted firmware could be installed on the simulated IoT device.

10. Rollback Protection

Rollback protection is a security mechanism that prevents IoT devices from installing older firmware versions after a newer version has already been deployed. This protection is important because attackers may attempt to downgrade devices to older firmware versions that contain known vulnerabilities.

The rollback protection mechanism implemented in this project ensures that only firmware versions equal to or newer than the currently installed version can be accepted for installation.

10.2 Objective

The objectives of implementing rollback protection were:

- Prevent firmware downgrade attacks
- Maintain the latest secure firmware version
- Block installation of outdated firmware
- Improve firmware lifecycle security
- Protect devices from known vulnerabilities in older releases

10.3 Firmware Version Simulation

To simulate firmware version validation, two version files were created.

Current installed firmware version:

```
echo "2.0" > firmware/current_version.txt
```

Incoming firmware update version:

```
echo "1.0" > firmware/update_version.txt
```

Purpose:

File	Description
current_version.txt	Currently installed firmware version
update_version.txt	Incoming firmware update version

In this test scenario, the incoming firmware version was intentionally set lower than the installed version to simulate a rollback attack.

10.4 Rollback Protection Implementation

A Python script was developed to compare firmware versions and determine whether the incoming update should be accepted or rejected.

Script File:

scripts/rollback_protection.py

Main Functions:

- Read installed firmware version
- Read incoming firmware version
- Compare version numbers
- Detect downgrade attempts
- Reject unauthorized firmware updates

The script automatically generates a security alert when a rollback attempt is detected.

10.5 Rollback Validation Test

The rollback protection script was executed using:

```
python scripts/rollback_protection.py
```

Observed Output:

Current	Installed	Firmware	Version:	2.0
Incoming		Firmware	Version:	1.0
[SECURITY				ALERT]
Rollback		attack		detected!
Firmware update rejected.				

Figure 10.1: Rollback Attack Detection

```
(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$ python scripts/rollback_protection.py
Current Installed Firmware Version: 2.0
Incoming Firmware Version: 1.0

[SECURITY ALERT]
Rollback attack detected!
Firmware update rejected.

(venv)-(kali㉿kali)-[~/secure-iot-firmware~]
$
```

10.6 Security Analysis

The rollback protection mechanism successfully identified that the incoming firmware version was older than the installed version.

The update was automatically rejected to prevent installation of potentially vulnerable firmware.

This demonstrates:

- Firmware version validation
- Downgrade attack prevention
- Secure firmware lifecycle management
- Protection against known vulnerabilities

Without rollback protection, attackers could intentionally install older firmware versions containing exploitable security weaknesses.

10.7 Testing Results

Test Scenario	Expected Result	Actual Result	Status
Incoming Version = Current Version	Allow Update	Allowed	Pass
Incoming Version > Current Version	Allow Update	Allowed	Pass
Incoming Version < Current Version	Reject Update	Rejected	Pass
Downgrade Attack Simulation	Detect Attack	Detected Successfully	Pass

10.8 Conclusion

The rollback protection mechanism successfully prevented firmware downgrade attacks by validating firmware version numbers before installation. The implementation ensured that outdated firmware could not replace newer secure versions, thereby improving overall firmware security and maintaining device integrity.

11. Recovery Mode

11.1 Overview

Recovery mode is an important security feature that enables an IoT device to restore a trusted firmware version when a firmware update fails or when firmware verification detects corruption or unauthorized modifications.

The recovery mechanism helps maintain device availability and prevents permanent device failure caused by corrupted, incomplete, or malicious firmware updates.

In this project, a recovery workflow was implemented to automatically restore a backup firmware image whenever firmware verification fails.

11.2 Objective

The objectives of implementing recovery mode were:

- Restore trusted firmware after failed updates
- Prevent device malfunction due to corrupted firmware
- Improve system resilience and reliability
- Maintain device availability
- Provide secure failure handling

11.3 Backup Firmware Creation

A backup firmware image was created to simulate a trusted firmware version available for recovery operations. The following command was used:

```
cp firmware/firmware_v1.bin firmware/backup_firmware.bin
```

Purpose:

File	Description
firmware_v1.bin	Original firmware
backup_firmware.bin	Trusted recovery firmware

The backup firmware serves as a fallback image that can be restored when verification failures occur.

11.4 Recovery Mode Implementation

A Python script was developed to simulate automatic recovery operations.

Script File:

```
scripts/recovery_mode.py
```

Main Features:

- Detect firmware verification failure
- Trigger recovery mode
- Restore backup firmware
- Generate recovery status messages
- Simulate secure firmware restoration

The script restores a trusted firmware copy whenever a verification failure condition is detected.

11.5 Recovery Validation Test

The recovery mode implementation was tested by simulating a firmware verification failure.

The recovery script was executed using:

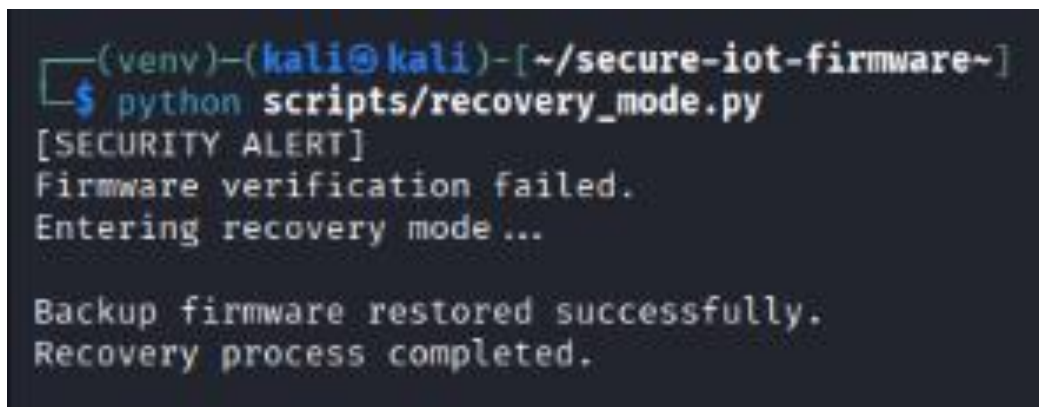
```
python scripts/recovery_mode.py
```

Observed Output:

```
[SECURITY ALERT]
Firmware verification failed.
Entering recovery mode...

Backup firmware restored successfully.
Recovery process completed.
```

Figure 11.1: Recovery Mode Execution

A screenshot of a terminal window with a dark background and light-colored text. The prompt shows the user is in a virtual environment named 'venv' on a 'kali' machine, in the directory '~/secure-iot-firmware~'. The user has executed the command 'python scripts/recovery_mode.py'. The output of the script is displayed below the command: '[SECURITY ALERT]', 'Firmware verification failed.', 'Entering recovery mode ...', 'Backup firmware restored successfully.', and 'Recovery process completed.'

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ python scripts/recovery_mode.py
[SECURITY ALERT]
Firmware verification failed.
Entering recovery mode ...

Backup firmware restored successfully.
Recovery process completed.
```

11.6 Recovery Verification

After successful execution of the recovery script, a restored firmware image was generated.

Generated File: `restored_firmware.bin`

This file represents the recovered firmware image restored from the trusted backup copy.

The successful creation of the restored firmware confirms that the recovery process completed successfully.

11.7 Security Analysis

The recovery mechanism successfully restored a trusted firmware image after a simulated verification failure.

The implementation demonstrated:

- Automatic recovery activation
- Secure firmware restoration
- Failure handling
- Device resilience
- Operational continuity

Recovery mechanisms are critical in IoT environments because firmware failures can render devices unavailable or unusable.

By maintaining a trusted backup firmware image, the device can continue operating even when update failures occur.

11.8 Testing Results

Test Scenario	Expected Result	Actual Result	Status
Firmware Verification Failure	Trigger Recovery Mode	Recovery Activated	Pass
Backup Firmware Availability	Restore Backup	Restored Successfully	Pass
Recovery Workflow Execution	Complete Recovery	Completed Successfully	Pass
Firmware Restoration	Generate Restored Firmware	Successful	Pass

11.9 Conclusion

The recovery mode implementation successfully demonstrated secure firmware restoration following a verification failure. The recovery workflow improved system resilience by ensuring that a trusted firmware image could be restored whenever update validation failed.

The implementation supports secure firmware lifecycle management and helps maintain device availability during firmware update failures.

12. RBAC Implementation

12.1 Overview

Role-Based Access Control (RBAC) is a security mechanism that restricts system access based on assigned user roles and permissions. RBAC ensures that users can only perform actions that are authorized for their role, reducing the risk of unauthorized modifications and improving security governance.

In this project, RBAC was implemented to control access to firmware update management functions and simulate secure administrative operations within an IoT firmware update environment.

12.2 Objective

The objectives of implementing RBAC were:

- Restrict access to sensitive firmware management operations
- Enforce the principle of least privilege
- Prevent unauthorized firmware modifications
- Improve security governance
- Separate administrative responsibilities

12.3 Implemented User Roles

The following roles were implemented in the system:

Role	Purpose
Administrator	Full firmware management and user administration
Operator	Update scheduling and monitoring
Viewer	Read-only access to logs and status information

Each role was assigned specific permissions according to its responsibilities.

12.4 RBAC Script Implementation

A Python-based RBAC simulation was developed to demonstrate role assignment and permission enforcement.

Script File:

scripts/rbac_simulation.py

Main Functions:

- User identification
- Role assignment
- Permission mapping
- Access validation
- Unauthorized access handling

The implementation verifies user roles and displays the permissions available to each authorized user.

12.5 Permission Mapping

Administrator Permissions

- Upload firmware
- Approve firmware updates
- View logs
- Manage users

Operator Permissions

- Schedule updates
- Monitor update status
- View logs

Viewer Permissions

- View logs only

This structure ensures that users receive only the permissions necessary for their assigned responsibilities.

12.6 RBAC Validation Testing

The RBAC implementation was tested using multiple user roles.

The script was executed using:

```
python scripts/rbac_simulation.py
```

Administrator Test

Input:

admin

Result:

Assigned	Role:	Administrator
Allowed		Permissions:
-	upload	firmware
-	approve	update
-	view	logs
- manage users		

Operator Test

Input:

operator

Result:

Assigned	Role:	Operator
Allowed		Permissions:
-	schedule	updates
- view logs		

Viewer Test

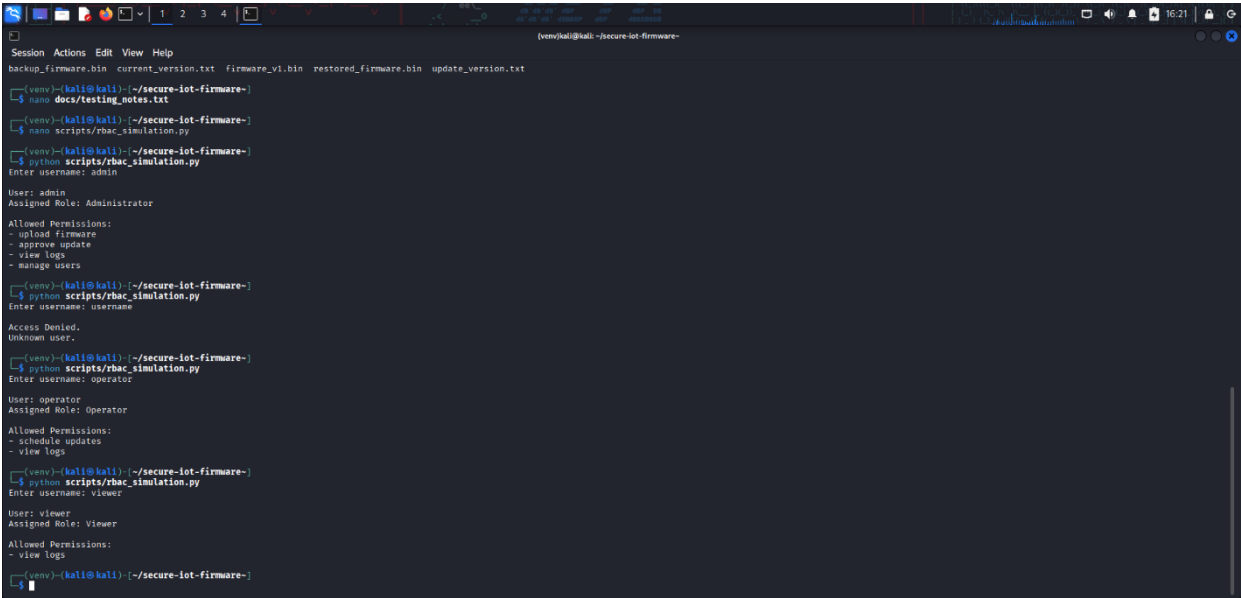
Input:

viewer

Result:

Assigned	Role:	Viewer
Allowed		Permissions:
- view logs		

Figure 12.1: RBAC Role Validation



12.7 Unauthorized Access Testing

An unauthorized user test was performed by entering an unknown username.

Input:

hacker

Observed Output:

Access	Denied.
Unknown user.	

The system successfully rejected the unauthorized user and prevented access to firmware management functions.

Figure 12.2: Unauthorized User Rejection

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ python scripts/rbac_simulation.py
Enter username: username

Access Denied.
Unknown user.
```

12.8 Security Analysis

The RBAC implementation successfully enforced role separation and access restrictions.

The implementation demonstrated:

- Least privilege enforcement
- Permission-based access control
- Unauthorized access prevention
- Administrative role separation
- Improved security governance

By restricting access according to user roles, the system reduces the risk of accidental or malicious firmware modifications.

12.9 Testing Results

Test Scenario	Expected Result	Actual Result	Status
Administrator Access	Full Permissions	Granted	Pass
Operator Access	Limited Permissions	Granted	Pass
Viewer Access	Read-Only Access	Granted	Pass
Unauthorized User	Access Denied	Denied Successfully	Pass

12.10 Conclusion

The RBAC implementation successfully demonstrated role-based authorization and access restriction for firmware update management. The system enforced the principle of least privilege by granting users only the permissions required for their role.

This implementation improved overall security governance and ensured that sensitive firmware management operations could only be performed by authorized users.

13. Audit Logging

13.1 Overview

Audit logging is an essential security mechanism used to record system activities, security events, user actions, and update operations. Audit logs provide visibility into system behavior and help security administrators monitor activities, investigate incidents, and detect unauthorized access attempts.

In this project, audit logging was implemented to record firmware update activities, authentication events, firmware download requests, and unauthorized access attempts within the secure firmware update environment.

13.2 Objective

The objectives of implementing audit logging were:

- Monitor firmware update activities
- Record security-related events
- Detect unauthorized access attempts
- Maintain an audit trail for investigations
- Support accountability and compliance requirements

13.3 Logging Implementation

The secure update server was configured to generate log entries whenever important events occurred.

The logging mechanism recorded activities such as:

- Firmware update requests
- Firmware download operations
- Authentication events
- Unauthorized access attempts
- Server activity

The generated logs were stored for monitoring and analysis purposes.

13.4 Logged Security Events

During testing, the following security events were successfully recorded:

Update Check Requests

When an IoT device checked for available updates, the request was recorded in the log system.

Example Event:

Update check request received

Authorized Firmware Downloads

When a valid authentication token was provided, the firmware download request was logged successfully.

Example Event:

Authorized firmware download request

Unauthorized Access Attempts

When a request was made without a valid authentication token, the server recorded the event as a security violation.

Example Event:

Unauthorized firmware access attempt

This allowed the system administrator to identify suspicious activity and investigate potential attacks.

13.5 Audit Log Validation Testing

The audit logging mechanism was validated by performing both authorized and unauthorized firmware access attempts.

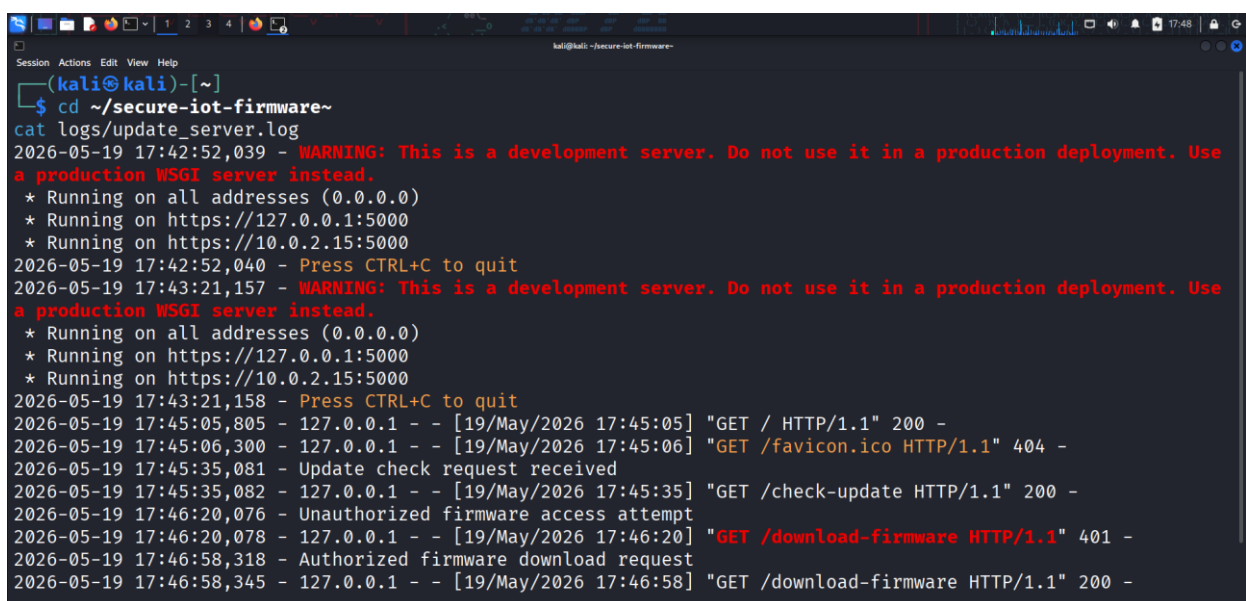
Test Activities:

1. Access update status endpoint
2. Download firmware using valid token
3. Attempt firmware download without authentication
4. Observe generated log entries

Observed Results:

Activity	Log Generated
Update Check Request	Yes
Authorized Firmware Download	Yes
Unauthorized Access Attempt	Yes
Server Activity	Yes

Figure 13.1: Audit Log Output



```
kali@kali:~/secure-iot-firmware$ cd ~/secure-iot-firmware~
kali@kali:~/secure-iot-firmware$ cat logs/update_server.log
2026-05-19 17:42:52,039 - WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on https://127.0.0.1:5000
* Running on https://10.0.2.15:5000
2026-05-19 17:42:52,040 - Press CTRL+C to quit
2026-05-19 17:43:21,157 - WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on https://127.0.0.1:5000
* Running on https://10.0.2.15:5000
2026-05-19 17:43:21,158 - Press CTRL+C to quit
2026-05-19 17:45:05,805 - 127.0.0.1 - - [19/May/2026 17:45:05] "GET / HTTP/1.1" 200 -
2026-05-19 17:45:06,300 - 127.0.0.1 - - [19/May/2026 17:45:06] "GET /favicon.ico HTTP/1.1" 404 -
2026-05-19 17:45:35,081 - Update check request received
2026-05-19 17:45:35,082 - 127.0.0.1 - - [19/May/2026 17:45:35] "GET /check-update HTTP/1.1" 200 -
2026-05-19 17:46:20,076 - Unauthorized firmware access attempt
2026-05-19 17:46:20,078 - 127.0.0.1 - - [19/May/2026 17:46:20] "GET /download-firmware HTTP/1.1" 401 -
2026-05-19 17:46:58,318 - Authorized firmware download request
2026-05-19 17:46:58,345 - 127.0.0.1 - - [19/May/2026 17:46:58] "GET /download-firmware HTTP/1.1" 200 -
```

13.6 Security Analysis

The audit logging implementation successfully captured critical firmware update activities and security events.

The logging mechanism provided:

- Activity monitoring
- Security event tracking
- Unauthorized access detection
- Incident investigation support
- Accountability for firmware operations

By maintaining a complete audit trail, administrators can review system activities and identify potential security issues affecting firmware updates.

13.7 Benefits of Audit Logging

The implemented audit logging system provides several security benefits:

- Improved visibility into firmware update operations
- Detection of suspicious activities
- Monitoring of authentication events
- Support for compliance requirements
- Enhanced incident response capabilities

Audit logs play an important role in maintaining trust and accountability within secure firmware update systems.

13.8 Testing Results

Test Scenario	Expected Result	Actual Result	Status
Update Check Request	Log Generated	Success	Pass
Authorized Download	Log Generated	Success	Pass
Unauthorized Access Attempt	Security Event Logged	Success	Pass
Server Activity Monitoring	Activity Recorded	Success	Pass

13.9 Conclusion

The audit logging implementation successfully recorded firmware update activities, authentication events, and unauthorized access attempts. The generated logs provided a reliable audit trail for monitoring, investigation, and accountability purposes.

The implementation enhanced the overall security of the firmware update framework by ensuring that important events were tracked and available for future analysis.

14. Testing & Validation

14.1 Overview

Testing and validation were performed to verify the effectiveness of the implemented security controls within the secure firmware update framework. The objective of testing was to ensure that firmware authenticity, integrity, access control, rollback protection, recovery mechanisms, and audit logging operated as expected under both normal and attack scenarios.

All tests were conducted in a controlled Kali Linux laboratory environment using open-source tools and custom Python scripts.

14.2 Test Environment

Component	Description
Operating System	Kali Linux
Programming Language	Python 3
Cryptographic Tool	OpenSSL
Web Framework	Flask
Communication Protocol	HTTPS/TLS
Testing Platform	VirtualBox

14.3 Test Case 1 – Firmware Signature Verification

Objective

To verify that authentic firmware signed with the RSA private key can be successfully validated using the corresponding public key.

Command Executed

```
openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
```

Expected Result

Verified OK

Actual Result

Verified OK

Status

Pass

Figure 14.1: Successful Firmware Signature Verification

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
  -signature signatures/firmware.sig firmware/firmware_v1.bin
Verified OK
```

Analysis

The firmware signature was successfully validated, confirming that the firmware originated from a trusted source and had not been modified after signing.

14.4 Test Case 2 – Firmware Tampering Detection

Objective

To verify that unauthorized firmware modifications are detected during signature verification.

Tampering Action

```
echo "Malicious Backdoor Injected" >> firmware/firmware_v1.bin
```

Verification Command

```
openssl dgst -sha256 -verify keys/public.pem \
  -signature signatures/firmware.sig firmware/firmware_v1.bin
```

Expected Result

Verification	failure
bad signature	

Actual Result

Verification	failure
bad signature	

Status: Pass

Figure 14.2: Firmware Tampering Detection

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
  -signature signatures/firmware.sig firmware/firmware_v1.bin
Verification failure
40D70B05187F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
40D70B05187F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:
```

Analysis

The system successfully detected unauthorized modifications to the firmware image and rejected the altered firmware.

14.5 Test Case 3 – Unauthorized Firmware Access

Objective

To verify that unauthorized users cannot download firmware from the update server.

Command Executed

```
curl -k https://127.0.0.1:5000/download-firmware
```

Expected Result

Unauthorized

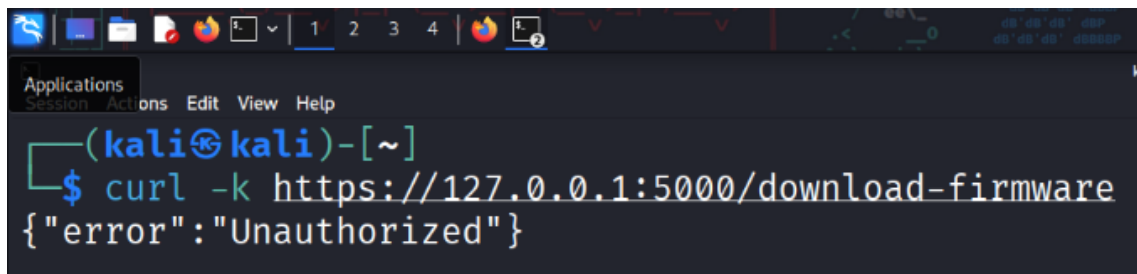
Actual Result

Unauthorized

Status

Pass

Figure 14.3: Unauthorized Access Attempt

A screenshot of a Kali Linux terminal window. The terminal shows the prompt `(kali㉿kali)-[~]` followed by the command `$ curl -k https://127.0.0.1:5000/download-firmware`. The output of the command is `{"error": "Unauthorized"}`. The terminal window has a menu bar with 'Applications', 'Session', 'Actions', 'Edit', 'View', and 'Help'. The background is dark with light-colored text.

Analysis

The token-based authentication mechanism successfully prevented unauthorized firmware downloads.

14.6 Test Case 4 – Rollback Attack Detection

Objective

To verify that older firmware versions are rejected.

Test Input

Current Version:

2.0

Incoming Version:

1.0

Command Executed

python scripts/rollback_protection.py

Actual Result

[SECURITY		ALERT]
Rollback	attack	detected!
Firmware update rejected.		

Status

Pass

Figure 14.4: Rollback Attack Detection

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]  
$ python scripts/rollback_protection.py  
Current Installed Firmware Version: 2.0  
Incoming Firmware Version: 1.0  
  
[SECURITY ALERT]  
Rollback attack detected!  
Firmware update rejected.
```

Analysis

The rollback protection mechanism correctly identified a downgrade attempt and prevented installation of vulnerable firmware.

14.7 Test Case 5 – Recovery Mode Validation

Objective

To verify that the recovery mechanism restores backup firmware when verification fails.

Command Executed

```
python scripts/recovery_mode.py
```

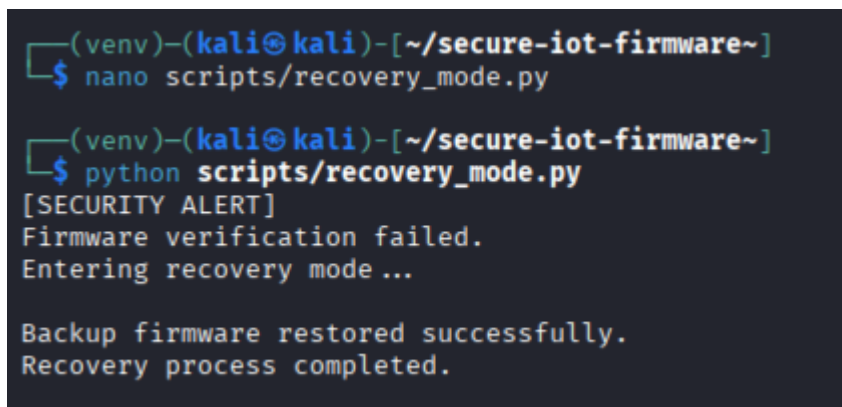
Actual Result

[SECURITY			ALERT]
Firmware	verification		failed.
Entering	recovery		mode...
Backup	firmware	restored	successfully.
Recovery process completed.			

Status

Pass

Figure 14.5: Recovery Mode Execution



```
(venv)-(kali㉿kali)-[~/secure-iot-firmware~]  
$ nano scripts/recovery_mode.py  
  
(venv)-(kali㉿kali)-[~/secure-iot-firmware~]  
$ python scripts/recovery_mode.py  
[SECURITY ALERT]  
Firmware verification failed.  
Entering recovery mode ...  
  
Backup firmware restored successfully.  
Recovery process completed.
```

Analysis

The recovery mechanism successfully restored a trusted firmware image and maintained system availability.

14.8 Test Case 6 – RBAC Validation

Objective

To verify that access permissions are enforced according to assigned roles.

Command Executed

python scripts/rbac_simulation.py

Tested Roles

- Administrator
- Operator
- Viewer
- Unauthorized User

Actual Result

Access Denied.
Unknown user.

Status

Pass

Figure 14.6: RBAC Validation

```
(venv)kali@kali: ~/secure-iot-firmware-
Session Actions Edit View Help
backup_firmware.bin current_version.txt firmware_v1.bin restored_firmware.bin update_version.txt
(venv)kali@kali:~/secure-iot-firmware-
$ nano docs/testing_notes.txt
(venv)kali@kali:~/secure-iot-firmware-
$ nano scripts/rbac_simulation.py
(venv)kali@kali:~/secure-iot-firmware-
$ python scripts/rbac_simulation.py
Enter username: admin
User: admin
Assigned Role: Administrator
Allowed Permissions:
- upload firmware
- approve update
- view logs
- manage users
(venv)kali@kali:~/secure-iot-firmware-
$ python scripts/rbac_simulation.py
Enter username: username
Access Denied.
Unknown user.
(venv)kali@kali:~/secure-iot-firmware-
$ python scripts/rbac_simulation.py
Enter username: operator
User: operator
Assigned Role: Operator
Allowed Permissions:
- schedule updates
- view logs
(venv)kali@kali:~/secure-iot-firmware-
$ python scripts/rbac_simulation.py
Enter username: viewer
User: viewer
Assigned Role: Viewer
Allowed Permissions:
- view logs
(venv)kali@kali:~/secure-iot-firmware-
$
```

Analysis

The RBAC implementation successfully enforced role separation and prevented unauthorized access.

14.9 Overall Testing Summary

Test Case	Result	Status
Firmware Signature Verification	Successful	Pass
Firmware Tampering Detection	Detected	Pass
Unauthorized Access Attempt	Blocked	Pass
Rollback Attack Detection	Prevented	Pass
Recovery Mode Validation	Successful	Pass
RBAC Validation	Successful	Pass

14.10 Conclusion

The testing and validation process confirmed that the implemented secure firmware update framework successfully protected against unauthorized firmware modification, rollback attacks, unauthorized access attempts, and update failures. All security controls performed as expected, demonstrating the effectiveness of the proposed secure firmware update mechanism for IoT devices.

15. Compliance Mapping

15.1 Overview

Compliance mapping was performed to align the implemented secure firmware update mechanism with recognized cybersecurity standards and security best practices. The objective was to ensure that the security controls implemented in the project support industry recommendations for firmware security, secure communications, access control, and incident monitoring.

The implementation was evaluated against:

- NIST IoT Cybersecurity Guidelines
- IoT Security Foundation Recommendations
- GDPR Security Principles
- HIPAA Security Requirements

15.2 NIST IoT Cybersecurity Alignment

The National Institute of Standards and Technology (NIST) provides cybersecurity guidance for IoT systems to improve device security, resilience, and risk management. The implemented security controls were mapped to relevant NIST cybersecurity objectives.

Table 15.1: NIST IoT Security Mapping

Implemented Feature	NIST Security Objective
RSA Firmware Signing	Device Integrity Protection
SHA-256 Verification	Firmware Integrity Validation
HTTPS/TLS Communication	Secure Data Transmission
Authentication Controls	Unauthorized Access Prevention
RBAC Implementation	Access Control Enforcement
Audit Logging	Security Monitoring
Rollback Protection	Secure Firmware Lifecycle Management
Recovery Mode	Resilience and Recovery

Analysis: The implemented framework supports core NIST security objectives by protecting firmware authenticity, enforcing access control, securing communications, and maintaining system resilience through recovery mechanisms.

15.3 IoT Security Foundation Alignment

The IoT Security Foundation provides best practice recommendations for securing IoT devices throughout their lifecycle.

Table 15.2: IoT Security Foundation Mapping

Implemented Feature	Security Benefit
Secure Firmware Signing	Trusted Firmware Validation
Firmware Verification	Tampering Prevention
HTTPS/TLS Communication	Secure Firmware Distribution
Authentication Mechanism	Authorized Device Access
Recovery Mode	Operational Resilience
Rollback Protection	Downgrade Attack Prevention
RBAC	Administrative Security Control
Audit Logging	Incident Investigation Support

Analysis

The implementation follows several key IoT Security Foundation recommendations by ensuring firmware authenticity, protecting update channels, and implementing secure recovery and monitoring capabilities.

15.4 GDPR Security Considerations

The General Data Protection Regulation (GDPR) requires organizations to implement appropriate technical and organizational measures to protect personal data and maintain system security.

Although this project does not directly process personal data, several implemented controls support GDPR security principles.

Implemented GDPR-Related Controls

- HTTPS/TLS encrypted communication
- Role-Based Access Control (RBAC)
- Audit logging and monitoring
- Firmware integrity validation
- Authentication mechanisms

Security Benefits

These controls help support:

- Confidentiality
- Integrity
- Accountability
- Secure system operation

By preventing unauthorized modifications and securing update communications, the framework contributes to a secure IoT operating environment.

15.5 HIPAA Security Considerations

The Health Insurance Portability and Accountability Act (HIPAA) requires healthcare systems to implement safeguards that protect electronic information and maintain system integrity.

The implemented framework supports several security concepts commonly associated with HIPAA requirements.

Implemented HIPAA-Related Controls

- Access control through RBAC
- Audit logging
- Secure communication using HTTPS/TLS
- Firmware integrity verification
- Authentication mechanisms

Security Benefits

These controls help provide:

- Controlled system access
- Activity monitoring
- Secure data transmission
- Protection against unauthorized modifications

Such controls are important for connected medical devices and healthcare-related IoT environments.

15.6 Open-Source Security Technologies Used

The project was implemented entirely using open-source technologies as required by the project guidelines.

Table 15.3: Open-Source Technologies

Technology	Purpose
OpenSSL	RSA Signing and SHA-256 Verification
Python 3	Automation and Validation Scripts
Flask	Secure Update Server
PyOpenSSL	TLS Support
HTTPS/TLS	Secure Communication
Kali Linux	Development and Testing Environment
VirtualBox	Controlled Lab Environment

15.7 Compliance Benefits

The implemented framework provides the following compliance-related benefits:

- Secure firmware authentication
- Firmware integrity validation
- Encrypted update communication
- Controlled administrative access
- Audit trail generation
- Downgrade attack prevention
- Recovery and resilience support
- Security monitoring and accountability

These features contribute to a more secure firmware update process and align with industry-recognized cybersecurity best practices.

15.8 Conclusion

The compliance mapping exercise demonstrated that the implemented secure firmware update framework aligns with several important cybersecurity recommendations and security principles. The combination of cryptographic verification, secure communication, access control, audit logging, rollback protection, and recovery mechanisms provides a strong foundation for secure firmware management in IoT environments.

16. CONCLUSION AND FUTURE ENHANCEMENTS

16.1 Conclusion

The objective of this project was to design and implement a secure firmware update mechanism for IoT devices that protects against unauthorized firmware installation, firmware tampering, downgrade attacks, and insecure update distribution. The project successfully demonstrated how cryptographic verification, secure communication, access control, and recovery mechanisms can be combined to improve firmware security.

During implementation, RSA-2048 digital signatures were used to authenticate firmware updates, while SHA-256 hashing was used to verify firmware integrity. A Flask-based secure update server was developed using HTTPS/TLS encryption to provide secure firmware distribution and protect update communications from unauthorized interception.

Several security mechanisms were successfully implemented and validated, including firmware signature verification, tampering detection, rollback protection, recovery mode, Role-Based Access Control (RBAC), authentication controls, and audit logging. Testing confirmed that the system was capable of detecting modified firmware, preventing unauthorized access, blocking downgrade attacks, and restoring trusted firmware when verification failures occurred.

The project also demonstrated alignment with NIST IoT Cybersecurity Guidelines, IoT Security Foundation recommendations, GDPR security principles, and HIPAA-oriented security practices. The implementation provided a practical simulation of a secure firmware update ecosystem and highlighted the importance of security controls throughout the firmware update lifecycle.

Overall, the project successfully achieved its objectives and demonstrated how secure firmware management can significantly improve the security and reliability of IoT devices.

16.2 Key Achievements

The following security features were successfully implemented and tested:

- RSA-2048 firmware signing and verification
- SHA-256 integrity validation
- HTTPS/TLS secure firmware distribution
- Token-based authentication
- Firmware tampering detection
- Rollback attack prevention
- Recovery mode implementation
- Role-Based Access Control (RBAC)
- Audit logging and monitoring
- Compliance mapping and security standards alignment

These features collectively improved firmware security, update integrity, and operational resilience.

16.3 Future Enhancements

Although the implemented framework successfully demonstrated secure firmware update concepts, several enhancements can be introduced in future versions to provide additional security and enterprise-level functionality.

Secure Boot Integration

Future implementations can integrate MCUboot or U-Boot with verified boot functionality to validate firmware signatures before execution and prevent unauthorized firmware from loading during device startup.

Hardware Security Module (HSM) Integration

Private signing keys can be stored in Hardware Security Modules (HSMs) or secure key vaults to improve key protection and reduce the risk of key compromise.

OTA Firmware Deployment Platforms

Open-source OTA platforms such as Mender or Eclipse hawkBit can be integrated to provide enterprise-grade firmware deployment and lifecycle management.

Delta Firmware Updates

Delta update mechanisms can be implemented to reduce bandwidth consumption by transmitting only modified portions of firmware instead of complete firmware images.

Real IoT Device Deployment

Future testing can be performed using physical IoT devices such as Raspberry Pi, ESP32, or STM32-based systems to validate the framework in real-world environments.

Multi-Factor Authentication (MFA)

Administrative access to firmware management systems can be strengthened through multi-factor authentication mechanisms.

SIEM and Advanced Monitoring

Integration with ELK Stack or Security Information and Event Management (SIEM) platforms can improve security monitoring, threat detection, and incident response capabilities.

Automated Security Reporting

Automated compliance reporting and firmware security assessment dashboards can be developed to improve visibility into firmware update operations.

16.4 Final Remarks

Firmware security is a critical component of modern IoT cybersecurity. As connected devices continue to grow across industries, secure firmware update mechanisms become essential for maintaining device integrity, preventing cyberattacks, and ensuring long-term operational reliability.

This project provided valuable practical experience in cryptography, secure communications, authentication, access control, recovery mechanisms, and firmware security management. The implemented framework demonstrates how multiple security controls can work together to create a trusted firmware update process and protect IoT devices against common security threats.

17. References

The following references were used during the design, implementation, testing, and documentation of the Secure Firmware Update Mechanism for IoT Devices project.

1. OpenSSL Project. *OpenSSL Documentation*. Available at: <https://www.openssl.org/>
2. Python Software Foundation. *Python 3 Documentation*. Available at: <https://docs.python.org/3/>
3. Flask Documentation. *Flask Web Framework*. Available at: <https://flask.palletsprojects.com/>
4. National Institute of Standards and Technology (NIST). *Considerations for Managing Internet of Things (IoT) Cybersecurity and Privacy Risks*. Available at: <https://www.nist.gov/>
5. National Institute of Standards and Technology (NIST). *Cybersecurity Framework (CSF)*. Available at: <https://www.nist.gov/cyberframework>
6. IoT Security Foundation. *IoT Security Compliance Framework*. Available at: <https://www.iotsecurityfoundation.org/>
7. MCUboot Project. *Open Source Secure Bootloader*. Available at: <https://www.mcuboot.com/>
8. U-Boot Project. *Universal Boot Loader Documentation*. Available at: <https://www.u-boot.org/>
9. HashiCorp Vault Documentation. *Secrets and Key Management*. Available at: <https://www.vaultproject.io/>
10. Mender.io Documentation. *Open Source OTA Update Management*. Available at: <https://mender.io/>
11. Eclipse hawkBit Documentation. *Software Update Management for IoT*. Available at: <https://www.eclipse.org/hawkbite/>
12. OWASP Foundation. *IoT Security Guidance*. Available at: <https://owasp.org/>
13. European Union. *General Data Protection Regulation (GDPR)*. Available at: <https://gdpr.eu/>
14. U.S. Department of Health & Human Services. *HIPAA Security Rule*. Available at: <https://www.hhs.gov/hipaa/>
15. Oracle VM VirtualBox Documentation. Available at: <https://www.virtualbox.org/>
16. Kali Linux Documentation. Available at: <https://www.kali.org/docs/>

APPENDICES

Appendix A – Important Commands Used

Firmware Creation

```
echo "Firmware Version 1.0 - Secure IoT Device" > firmware/firmware_v1.bin
```

RSA Private Key Generation

```
openssl genrsa -out keys/private.pem 2048
```

RSA Public Key Generation

```
openssl rsa -in keys/private.pem -pubout -out keys/public.pem
```

SHA-256 Hash Generation

```
sha256sum firmware/firmware_v1.bin
```

Firmware Signing

```
openssl dgst -sha256 -sign keys/private.pem \  
-out signatures/firmware.sig firmware/firmware_v1.bin
```

Firmware Verification

```
openssl dgst -sha256 -verify keys/public.pem \  
-signature signatures/firmware.sig firmware/firmware_v1.bin
```

Firmware Tampering Simulation

```
echo "Malicious Backdoor Injected" >> firmware/firmware_v1.bin
```

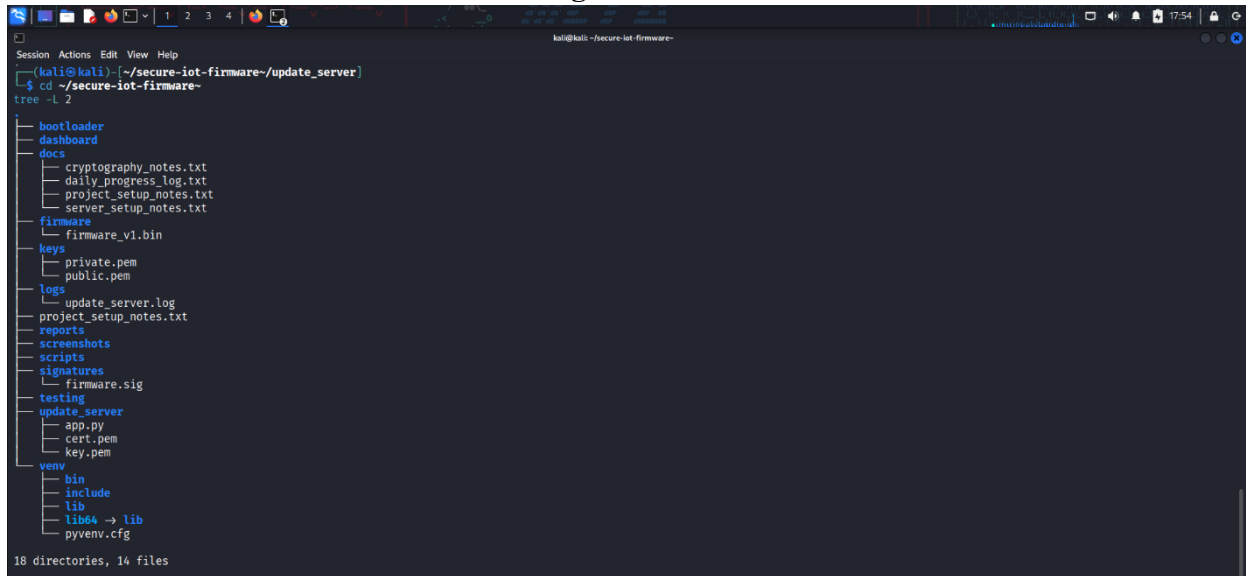
Secure Firmware Download

```
curl -k -H "Authorization: secureiottoken" \  
https://127.0.0.1:5000/download-firmware
```

Appendix B – Project Screenshots

Include the screenshots collected during implementation and testing.

Figure B.1

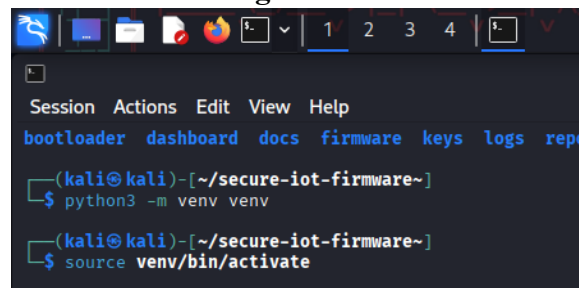


```
Session Actions Edit View Help
(kali@kali)-[~/secure-iot-firmware/update_server]
$ cd ~/secure-iot-firmware-
tree -L 2
.
├── bootloader
├── dashboard
├── docs
│   ├── cryptography_notes.txt
│   ├── daily_progress_log.txt
│   ├── project_setup_notes.txt
│   └── server_setup_notes.txt
├── firmware
│   └── firmware_v1.bin
├── keys
│   ├── private.pem
│   └── public.pem
├── logs
│   └── update_server.log
├── project_setup_notes.txt
├── reports
├── screenshots
├── scripts
├── signatures
│   └── firmware.sig
├── testing
├── update_server
│   ├── app.py
│   ├── cert.pem
│   └── key.pem
└── venv
    ├── bin
    ├── include
    ├── lib
    ├── lib64 → lib
    └── pyvenv.cfg

18 directories, 14 files
```

Project Structure

Figure B.2



```
Session Actions Edit View Help
bootloader dashboard docs firmware keys logs repo

(kali@kali)-[~/secure-iot-firmware~]
$ python3 -m venv venv

(kali@kali)-[~/secure-iot-firmware~]
$ source venv/bin/activate
```

Python Environment Setup

Figure B.3



```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl genrsa -out keys/private.pem 2048

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl rsa -in keys/private.pem -pubout -out keys/public.pem
writing RSA key

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ ls keys
private.pem  public.pem

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$
```

RSA Key Generation

Figure B.4

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ sha256sum firmware/firmware_v1.bin
cd2cab967e9f941a5cb1151ac3c27126e1f39d0c7edf61ff8c806993c9c44145  firmware/firmware_v1.bin

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -sign keys/private.pem \
-out signatures/firmware.sig firmware/firmware_v1.bin
```

SHA-256 Verification

Figure B.5

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
Verified OK
```

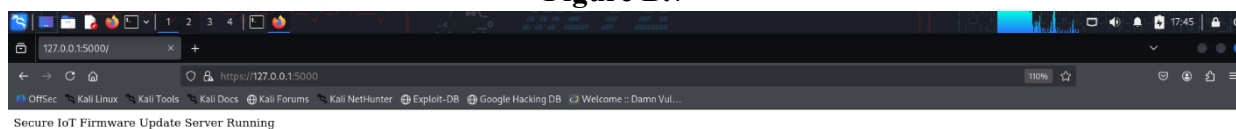
Firmware Signature Verification

Figure B.6

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ openssl dgst -sha256 -verify keys/public.pem \
-signature signatures/firmware.sig firmware/firmware_v1.bin
Verification failure
40070B05187F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
40070B05187F0000:error:1C800004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:
```

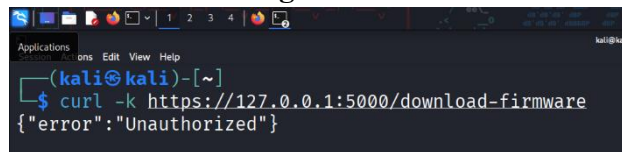
Tampered Firmware Detection

Figure B.7



HTTPS Update Server Running

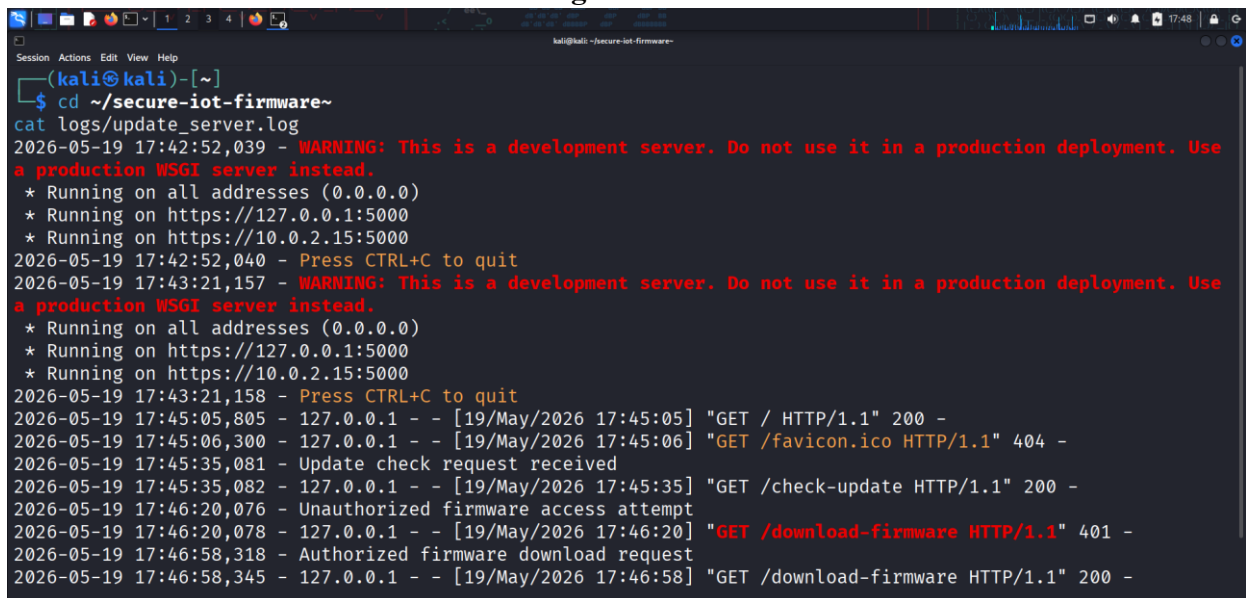
Figure B.8

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The user enters the command curl -k https://127.0.0.1:5000/download-firmware. The output is {"error": "Unauthorized"}.

```
(kali@kali)-[~]  
$ curl -k https://127.0.0.1:5000/download-firmware  
{"error": "Unauthorized"}
```

Unauthorized Access Attempt

Figure B.9

A terminal window showing the output of a web application. The user has navigated to ~/secure-iot-firmware and viewed the logs/update_server.log. The log contains several entries, including warnings about development servers, status messages about running on specific addresses, and HTTP request logs. Notable entries include an unauthorized firmware access attempt and a successful firmware download request.

```
(kali@kali)-[~]  
$ cd ~/secure-iot-firmware~  
cat logs/update_server.log  
2026-05-19 17:42:52,039 - WARNING: This is a development server. Do not use it in a production deployment. Use  
a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on https://127.0.0.1:5000  
* Running on https://10.0.2.15:5000  
2026-05-19 17:42:52,040 - Press CTRL+C to quit  
2026-05-19 17:43:21,157 - WARNING: This is a development server. Do not use it in a production deployment. Use  
a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on https://127.0.0.1:5000  
* Running on https://10.0.2.15:5000  
2026-05-19 17:43:21,158 - Press CTRL+C to quit  
2026-05-19 17:45:05,805 - 127.0.0.1 - - [19/May/2026 17:45:05] "GET / HTTP/1.1" 200 -  
2026-05-19 17:45:06,300 - 127.0.0.1 - - [19/May/2026 17:45:06] "GET /favicon.ico HTTP/1.1" 404 -  
2026-05-19 17:45:35,081 - Update check request received  
2026-05-19 17:45:35,082 - 127.0.0.1 - - [19/May/2026 17:45:35] "GET /check-update HTTP/1.1" 200 -  
2026-05-19 17:46:20,076 - Unauthorized firmware access attempt  
2026-05-19 17:46:20,078 - 127.0.0.1 - - [19/May/2026 17:46:20] "GET /download-firmware HTTP/1.1" 401 -  
2026-05-19 17:46:58,318 - Authorized firmware download request  
2026-05-19 17:46:58,345 - 127.0.0.1 - - [19/May/2026 17:46:58] "GET /download-firmware HTTP/1.1" 200 -
```

Audit Logging Output

Figure B.10

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ python scripts/rollback_protection.py
Current Installed Firmware Version: 2.0
Incoming Firmware Version: 1.0

[SECURITY ALERT]
Rollback attack detected!
Firmware update rejected.
```

Rollback Attack Detection

Figure B.11

```
(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ python scripts/recovery_mode.py
[SECURITY ALERT]
Firmware verification failed.
Entering recovery mode ...

Backup firmware restored successfully.
Recovery process completed.
```

Recovery Mode Execution

Figure B.12

```
$ python scripts/rbac_simulation.py
Enter username: admin
User: admin
Assigned Role: Administrator
Allowed Permissions:
- upload firmware
- approve update
- view logs
- manage users

(venv)-(kali@kali)-[~/secure-iot-firmware~]
$ python scripts/rbac_simulation.py
Enter username: username
Access Denied.
Unknown user.
```

RBAC Validation

Appendix C – Python Scripts

rollback_protection.py

Purpose:

Detects firmware downgrade attempts and prevents installation of older firmware versions.

recovery_mode.py

Purpose:

Restores backup firmware when verification fails.

rbac_simulation.py

Purpose:

Implements role-based access control for firmware management operations.

Appendix D – Testing Summary

Test Case	Result	Status
Firmware Verification	Successful	Pass
Tampered Firmware Detection	Successful	Pass
Unauthorized Access Prevention	Successful	Pass
Rollback Protection	Successful	Pass
Recovery Mode	Successful	Pass
RBAC Validation	Successful	Pass

Appendix E – Project Directory Structure

secure-iot-firmware/

├── bootloader/

├── dashboard/

├── docs/

├── firmware/

├── keys/

├── logs/

├── reports/

├── screenshots/

├── scripts/

├── signatures/

├── testing/

├── update_server/

└── README.md